

- **Convolutional Neural Network (CNN) Model for Face Recognition**

CNN model Developed for recognizing the 3 angles of Faces (Front, left, and Right) using the libraries Tensorflow and Keras. The Trained model are organized on folder Dataset which contains a labeled subfolders of each Face angle.

Model Architecture and Training

The model consists of three convolutional layers followed by max-pooling layers to extract the features from the images. These Layers are followed by fully connected layers for classification. The Final layer uses the softmax activation function to classify images into one of the three classes: front, left and right.

Steps in training:

1. Data preprocessing: using ImageDataGenerator to augment images by applying transformation such as rotations, rescaling, flipping to improve the model generalization.
2. Hyperparameters: we used an image size of 128x128, a batch 32 and trained the model for 10 epoch.

```
# CNN Model
model = Sequential([
    Conv2D(32, (3, 3), activation='relu', input_shape=(IMG_HEIGHT, IMG_WIDTH, 3)),
    MaxPooling2D(pool_size=(2, 2)),

    Conv2D(64, (3, 3), activation='relu'),
    MaxPooling2D(pool_size=(2, 2)),

    Conv2D(128, (3, 3), activation='relu'),
    MaxPooling2D(pool_size=(2, 2)),

    Flatten(),
    Dense(128, activation='relu'),
    Dropout(0.5),
    Dense(64, activation='relu'),
    Dropout(0.5),

    Dense(3, activation='softmax') # 3 classes: front, left, right
])

model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
```

Challenge and Solutions:

- **Challenge:** overfitting due to limited data.

- **Solution:** Implementation of Data Augmentation to artificially increase the dataset size, providing more variety in the training images
- Challenge: Imbalanced class distribution
 - Solution: The training generator was tuned to balance the classes during training

Evaluation of the CNN Model

1. Evaluation of the CNN Model

After training the CNN model for face recognition, the performance was evaluated using a separate validation dataset. The evaluation metrics used include validation loss and accuracy.

```
# Evaluate on validation set
val_loss, val_acc = model.evaluate(val_generator)
print(f"Validation Accuracy: {val_acc:.2f}")
```

Results: The validation accuracy reported was crucial in determining the model's effectiveness. A validation accuracy close to 1.0 indicates good performance, while a significant gap between training and validation accuracy may indicate overfitting.

Metrics Calculation

- **Precision:** Indicates the ratio of true positive predictions to the total predicted positives. It reflects the accuracy of the positive predictions.
- **Recall:** Represents the ratio of true positive predictions to the actual positives. It indicates the model's ability to identify all relevant instances.
- **F1 Score:** The harmonic mean of precision and recall, providing a single metric that balances both.
- **Accuracy:** The overall ratio of correctly predicted instances to total instances.

```

# Get test data generator
test_datagen = ImageDataGenerator(rescale=1./255)
test_generator = test_datagen.flow_from_directory(
    base_dir, # Same directory, but you should ideally have a separate test folder
    target_size=(IMG_HEIGHT, IMG_WIDTH),
    batch_size=BATCH_SIZE,
    class_mode='categorical',
    shuffle=False # Important to keep test data ordering the same
)

# Model predictions for metrics calculation
Y_pred = model.predict(test_generator)
y_pred = np.argmax(Y_pred, axis=1)
y_true = test_generator.classes

# Calculate precision, recall, and F1 score
precision = precision_score(y_true, y_pred, average='macro')
recall = recall_score(y_true, y_pred, average='macro')
f1 = f1_score(y_true, y_pred, average='macro')
accuracy = accuracy_score(y_true, y_pred)

print("Precision: ", precision)
print("Recall: ", recall)
print("F1 Score: ", f1)
print("Accuracy: ", accuracy)

```

• YOLOv8 for Face Model for Face Recognition

YoloV8 became a object detection was utilized to detect object in a custom dataset, it down to this path because we annotated the image faces and been set as Object, using the ultralytics library and Annotated in roboflow. The dataset was pre-labeled and the model was trained using transfer learning by fine tuning a pre trained YoloV8 model.

The validation accuracy provides a measure of how well the model is generalizing to unseen data. It is calculated after each epoch during the training process and indicates the percentage of correct predictions.

Model training and Architecture

1. **Data:** the configuration of dataset was defined in a data.yaml file contains path to training and validation of data.
2. **Training configuration:** Since our model was trained on CPU, the Images sizes are reduced to 416x416, batch size is 8 used to avoid memory issues and trained for epoch 10

```

# Load a pre-trained YOLOv8n model
model = YOLO('yolov8n.pt')

# Train the model on your custom dataset with CPU
model.train(
    data='dataset/data.yaml', # Path to your dataset YAML file
    epochs=10,                 # Adjust based on your needs
    imgsz=416,                 # Reduce the image size for faster CPU training
    batch=8,                   # Smaller batch size for CPU training
    device='cpu',              # Set device to CPU
    workers=4,                 # Use fewer workers for CPU training
)

```

Challenge and Solutions

- **Challenge:** Training on a CPU is slow.
 - **Solution:** Reduced image size, batch size, and number of epochs to accelerate training.
- **Challenge:** Balancing precision and recall.
 - **Solution:** Fine tuned the number of epochs and learning rate to improve the precision and recall, ensuring better detection accuracy.

Evaluation of the YOLOv8 Model

- **Accuracy:** This metric is crucial for object detection tasks and represents the average precision across different IoU (Intersection over Union) thresholds, typically measured at 50% and 50:95% (the latter being the average precision over multiple IoU thresholds).
- **Precision:** Indicates the ratio of true positive detections to the total detected instances. It reflects the accuracy of the model's detections.
- **Recall:** Represents the ratio of true positive detections to the total actual objects. It indicates the model's ability to detect all relevant objects.
- **F1 Score:** The harmonic mean of precision and recall, providing a single metric that balances both.

```

# Evaluate the model on the validation set
metrics = model.val()

# Access the evaluation metrics
precision = metrics.box.p # Precision per class
recall = metrics.box.r    # Recall per class
f1_score = metrics.box.f1  # F1 score per class
map50 = metrics.box.map50  # mAP@50

# Calculate averages
average_precision = np.mean(precision) if precision.size > 0 else 0
average_recall = np.mean(recall) if recall.size > 0 else 0
average_f1_score = np.mean(f1_score) if f1_score.size > 0 else 0
average_map50 = np.mean(map50) if map50.size > 0 else 0

# Print the results
print("Average Precision: ", average_precision)
print("Average Recall: ", average_recall)
print("Average F1 Score: ", average_f1_score)
print("Average Accuracy: ", average_map50)

```

- **Fast Region-based Convolutional Neural Network (FAST-R CNN) Model for Face Recognition**

The Fast-R algorithm is designed as a convolutional neural network (CNN) for image classification, particularly to recognize different orientations of faces (e.g., front, left, right). The model is developed using the Keras library within TensorFlow, employing a structured approach for training and evaluation.

Implementation Steps

1. **Dataset Preparation:**

- The dataset is organized in a directory structure where each subfolder corresponds to a class (front, left, right). This facilitates easy loading and processing.
- The ImageDataGenerator is used for preprocessing and data augmentation, enhancing the robustness of the model by introducing variations in the training images (e.g., rotation, width/height shifts).

```
# ImageDataGenerator for preprocessing and augmentation
train_datagen = ImageDataGenerator(
    rescale=1./255,
    rotation_range=20,
    width_shift_range=0.2,
    height_shift_range=0.2,
    horizontal_flip=True,
    validation_split=0.2 # Reserve 20% of data for validation
)
```

Model Architecture:

- The CNN architecture comprises several convolutional layers followed by pooling layers, culminating in fully connected layers. This structure is designed to extract spatial hierarchies from the images.
- Dropout layers are incorporated to reduce overfitting by randomly setting a fraction of input units to zero during training.

```
# FAST-R CNN Model
model = Sequential([
    Conv2D(32, (3, 3), activation='relu', input_shape=(IMG_HEIGHT, IMG_WIDTH, 3)),
    MaxPooling2D(pool_size=(2, 2)),

    Conv2D(64, (3, 3), activation='relu'),
    MaxPooling2D(pool_size=(2, 2)),

    Conv2D(128, (3, 3), activation='relu'),
    MaxPooling2D(pool_size=(2, 2)),

    Flatten(),
    Dense(128, activation='relu'),
    Dropout(0.5),
    Dense(64, activation='relu'),
    Dropout(0.5),

    Dense(3, activation='softmax') # 3 classes: test, valid, train
])
```

Model Compilation and Training:

- The model is compiled using the Adam optimizer and categorical cross-entropy loss function suitable for multi-class classification.
- The model is then trained using the training data generator, with validation metrics monitored to prevent overfitting.

```

model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])

# Train the model
history = model.fit(
    train_generator,
    epochs=EPOCHS,
    validation_data=val_generator
)

```

Evaluation:

- After training, the model is evaluated on a separate validation set. Metrics such as accuracy, precision, recall, and F1 score are computed to assess model performance.

```

# Evaluate on validation set
val_loss, val_acc = model.evaluate(val_generator)
print(f"Validation Accuracy: {val_acc:.2f}")

```

Fine-Tuning Steps

- **Hyperparameter Tuning:** Experimenting with different values for learning rates, batch sizes, and the number of epochs to find optimal configurations.
- **Data Augmentation:** Further augmenting the dataset with additional transformations to improve the model's ability to generalize.
- **Early Stopping:** Implementing early stopping to halt training when validation loss starts to increase, indicating potential overfitting.
- **Model Checkpointing:** Saving the best-performing model based on validation accuracy for later use.

Challenges Encountered and Solutions

- **Overfitting:** The model showed signs of overfitting early in training. This was mitigated by introducing dropout layers and using data augmentation to provide more diverse training samples.
- **Insufficient Data:** Limited training data led to poor model generalization. To address this, data augmentation was employed, and efforts were made to gather more images for each class.

Application or Software Development

Frameworks and Programming Languages:

- **Python:** Used for model development, training, and evaluation of CNN, FAST-R CNN and YOLO V8 models.
- **Laravel (PHP):** The backend framework used to build the application that interfaces with the trained models.
- **JavaScript/AJAX:** For real-time interaction with the models, such as capturing live images for facial recognition.

Architecture and Integration:

The application is structured around a client-server model where the web application communicates with the Laravel via AJAX requests to initiate predictions using the models. The backend invokes Python scripts to load the trained CNN, FAST-R CNN and YOLO v8 models and returns predictions.

Steps in Development:

1. **User Interface:** Developed using HTML/CSS with JavaScript to handle image uploads and capture live data.
2. **Backend Integration:**
 - **Laravel:** Acts as the server, responsible for handling user requests and passing images to the Python-based model scripts.
 - **Python Model Integration:** The Laravel controller runs the Python scripts using `escapeshellarg` and `exec` to pass inputs (image paths) and retrieve predictions from the CNN, FAST-R CNN and YOLO v8 models.

```
1 reference
private function runPythonScript($uploadedImagePath, $capturedImagePath): mixed
{
    $scriptPath = realpath(path: base_path(path: 'scripts' . DIRECTORY_SEPARATOR . 'cnn_predict.py'));
    if (!$scriptPath) {
        Log::error(message: 'Python script not found', context: ['script' => $scriptPath]);
        return ['status' => 'error', 'message' => 'Python script not found'];
    }
    $command = 'python ' . escapeshellarg(arg: $scriptPath) . ' ' .
        escapeshellarg(arg: public_path(path: $uploadedImagePath)) . ' ' .
        escapeshellarg(arg: public_path(path: $capturedImagePath));
    $output = [];
    $returnVar = 0;
    exec(command: $command . ' 2>&1', output: &$output, result_code: &$returnVar);
    $outputString = implode(separator: "\n", array: $output);

    $jsonString = $this->formatOutput(outputString: $outputString);
    if ($jsonString) {
        Log::error(message: 'Unexpected response from Python script', context: ['output' => $outputString]);
        return ['status' => 'error', 'message' => 'Unexpected response from the server. Please check the logs.'];
    }
    $decodedOutput = json_decode(json: $jsonString, associative: true);
    if (json_last_error() !== JSON_ERROR_NONE) {
        Log::error(message: 'JSON decoding error', context: ['output' => $jsonString, 'error' =>
            json_last_error_msg()]);
        return ['status' => 'error', 'message' => 'Error decoding JSON response from the server.'];
    }
    Log::info(message: 'Face scan completed', context: ['algorithm' => 'cnn', 'output' => $decodedOutput]);
    return $decodedOutput;
}
```



```
private function runRcnnPythonScript($uploadedImagePath, $capturedImagePath): mixed
{
    $scriptPath = realpath(path: base_path(path: 'scripts' . DIRECTORY_SEPARATOR . 'fastr_predict.py'));
    if (!$scriptPath) {
        Log::error(message: 'Python script not found', context: ['script' => $scriptPath]);
        return ['status' => 'error', 'message' => 'Python script not found'];
    }
    $command = 'python ' . escapeshellarg(arg: $scriptPath) . ' ' .
        escapeshellarg(arg: public_path(path: $uploadedImagePath)) . ' ' .
        escapeshellarg(arg: public_path(path: $capturedImagePath));
    $output = [];
    $returnVar = 0;
    exec(command: $command . ' 2>&1', output: &$output, result_code: &$returnVar);
    $outputString = implode(separator: "\n", array: $output);

    $jsonString = $this->extractJsonFromOutput(outputString: $outputString);
    if ($jsonString === false) {
        Log::error(message: 'Unexpected response from Python script', context: ['output' => $outputString]);
        return ['status' => 'error', 'message' => 'Unexpected response from the server. Please check the logs.'];
    }
    $decodedOutput = json_decode(json: $jsonString, associative: true);
    if (json_last_error() !== JSON_ERROR_NONE) {
        Log::error(message: 'JSON decoding error', context: ['output' => $jsonString, 'error' => json_last_error()]);
        return ['status' => 'error', 'message' => 'Error decoding JSON response from the server.'];
    }
    Log::info(message: 'Face scan completed', context: ['algorithm' => 'fastr', 'output' => $decodedOutput]);
    return $decodedOutput;
}
```

```
private function runYolocnnPythonScript($uploadedImagePath, $capturedImagePath): mixed
{
    $scriptPath = realpath(path: base_path(path: 'scripts' . DIRECTORY_SEPARATOR . 'yolo_predict.py'));
    if (!$scriptPath) {
        Log::error(message: 'Python script not found', context: ['script' => $scriptPath]);
        return ['status' => 'error', 'message' => 'Python script not found'];
    }
    $command = 'python ' . escapeshellarg(arg: $scriptPath) . ' ' . escapeshellarg(arg: public_path(path: $uploadedImagePath)) . ' ' .
        escapeshellarg(arg: public_path(path: $capturedImagePath));
    $output = [];
    $returnVar = 0;
    exec(command: $command . ' 2>&1', output: &$output, result_code: &$returnVar);
    $outputString = implode(separator: "\n", array: $output);

    $jsonString = $this->extractJsonFromOutput(outputString: $outputString);
    if ($jsonString === false) {
        Log::error(message: 'Unexpected response from Python script', context: ['output' => $outputString]);
        return ['status' => 'error', 'message' => 'Unexpected response from the server. Please check the logs.'];
    }
    $decodedOutput = json_decode(json: $jsonString, associative: true);
    if (json_last_error() !== JSON_ERROR_NONE) {
        Log::error(message: 'JSON decoding error', context: ['output' => $jsonString, 'error' => json_last_error()]);
        return ['status' => 'error', 'message' => 'Error decoding JSON response from the server.'];
    }
    Log::info(message: 'Face scan completed', context: ['algorithm' => 'yolo', 'output' => $decodedOutput]);
    return $decodedOutput;
}
```

3. Testing and Iteration:

- **Unit Testing:** Each component, including model prediction, image processing, and file handling, was tested individually.

- **Agile Principles:** Iterative development cycles were followed, where features were implemented, tested, and refined based on user feedback and performance metrics.
- **Debugging:** Continuous logging and error handling were added to the Python and Laravel layers to ensure smooth execution.

Challenges in Application Development:

- **Model Integration:** Ensuring that the TensorFlow and YOLO models, written in Python, seamlessly integrate with Laravel (written in PHP).
 - **Solution:** Leveraged the `escapeshellarg` and `exec` function in Laravel to run Python scripts and manage inputs/outputs via file-based communication.
- **Real-Time Performance:** Optimizing the response time of predictions, especially for face recognition using the CNN and FAST-R model.
 - **Solution:** Reduced image size and optimized the CNN and FAST-R model to make predictions faster without significantly compromising accuracy.

Debugging and Continuous Improvement:

- Debugging was carried out by logging the outputs of each stage (image preprocessing, prediction scores, etc.) and tracing errors back to either the model or data pipeline.